

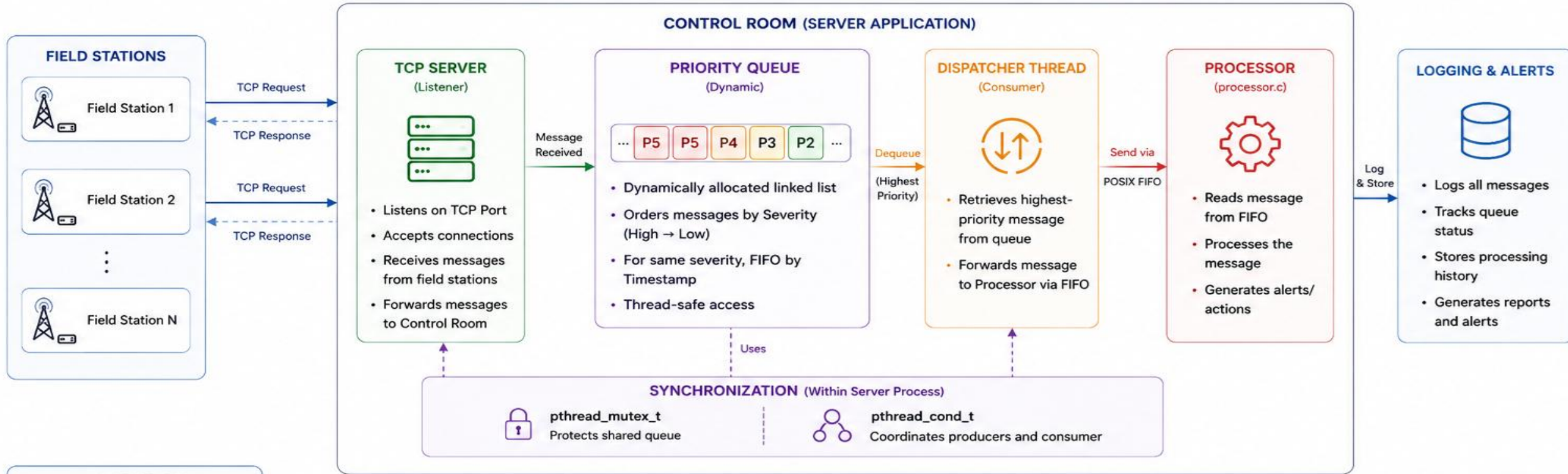
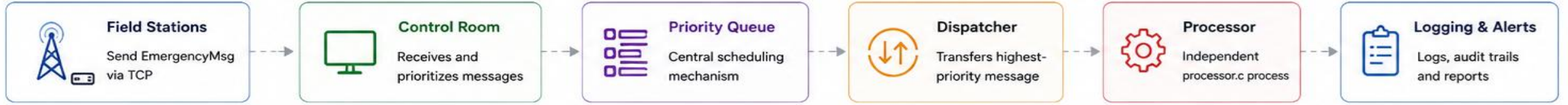
The background illustration depicts a control room for a disaster communication system. Several large monitors are visible. On the left, a map of Europe shows 'STATION 3' and 'STATION 4' with red location markers. A central monitor displays a '3D TO AIR' view of a disaster site with smoke and debris. To the right, another map shows 'ONLINE 112 ACTIVE' with red markers. A smaller screen shows 'ALERTS 24' with a red bar chart and a 'SEVERITY' table. Another screen shows 'SYSTEMS ONLINE STATUS' with CPU, MEM, and NET usage, 'MESSAGES TOTAL 256', and 'QUEUED 14'. In the foreground, several operators in white uniforms and helmets are working. One operator's vest is marked 'A3'. They are using handheld devices and a large control panel with many buttons and a small screen. The overall scene is a detailed, stylized illustration of emergency response technology.

Real-Time Disaster Communication System

A **C-based** real-time emergency communication platform with **TCP/IP**, **POSIX Threads**, and **IPC**.

Digvijay A (192320045)
Reshini Priya B (192320035)
Vishal J (192412433)

SYSTEM ARCHITECTURE



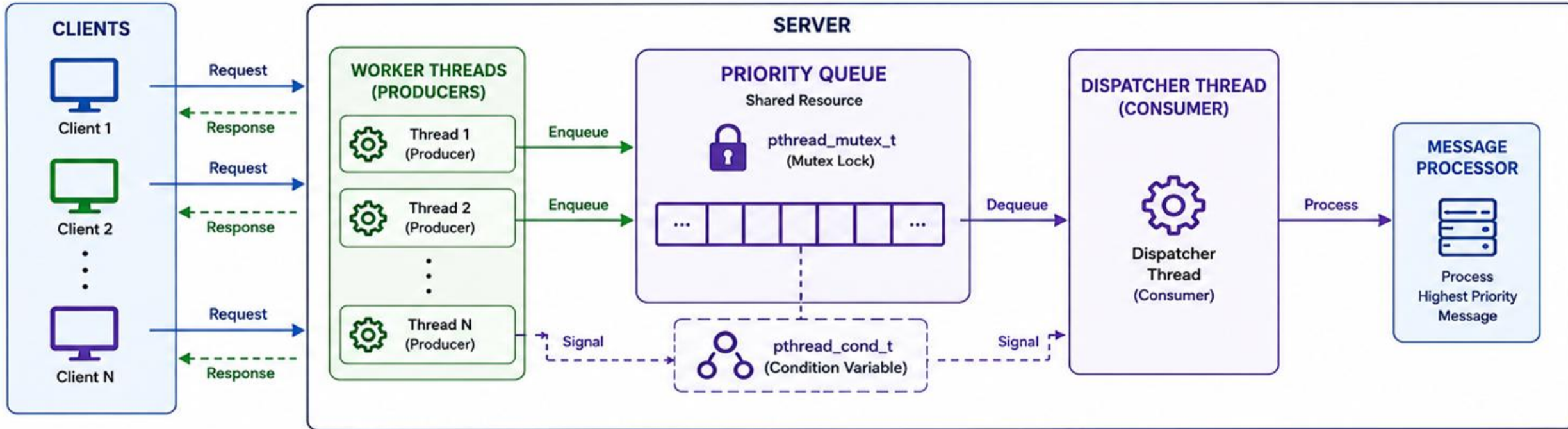
CONCURRENCY & SYNCHRONIZATION

THREAD MODEL

- One detached pthread per client
- Multiple clients handled simultaneously
- Worker threads act as **Producers**
- Dispatcher thread acts as **Consumer**

SYNCHRONIZATION PRIMITIVES

- pthread_mutex_t protects shared queue
- pthread_cond_t coordinates producers and consumer
- Race-condition-free shared data access



LEGEND

- Client Request / System Flow
- Enqueue (Produce)
- Dequeue (Consume)
- - - - - Response to Client
- - - - - Synchronization Signal

EXECUTION FLOW



DYNAMIC PRIORITY QUEUE

A linked list based priority queue where elements are ordered by severity (higher first). For same severity, FIFO (earlier timestamp first).

STRUCTURE

- Dynamically allocated linked list
- Pointer-based management
- Supports varying workloads

NODE FORMAT

Data / Message	Severity (1-5)	Timestamp (μs)	Next Pointer

PRIORITY RULES

- Severity levels: 1 (Low) – 5 (High)
- Higher severity → Higher priority
- If severity is same → Earlier timestamp first (FIFO)



ORDERING EXPLANATION

System Alert (5) came before Power Failure (5) because its timestamp is earlier.
Then comes severity 4, then 3, then 2.

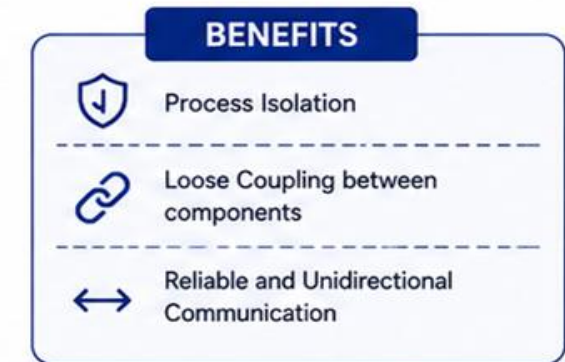
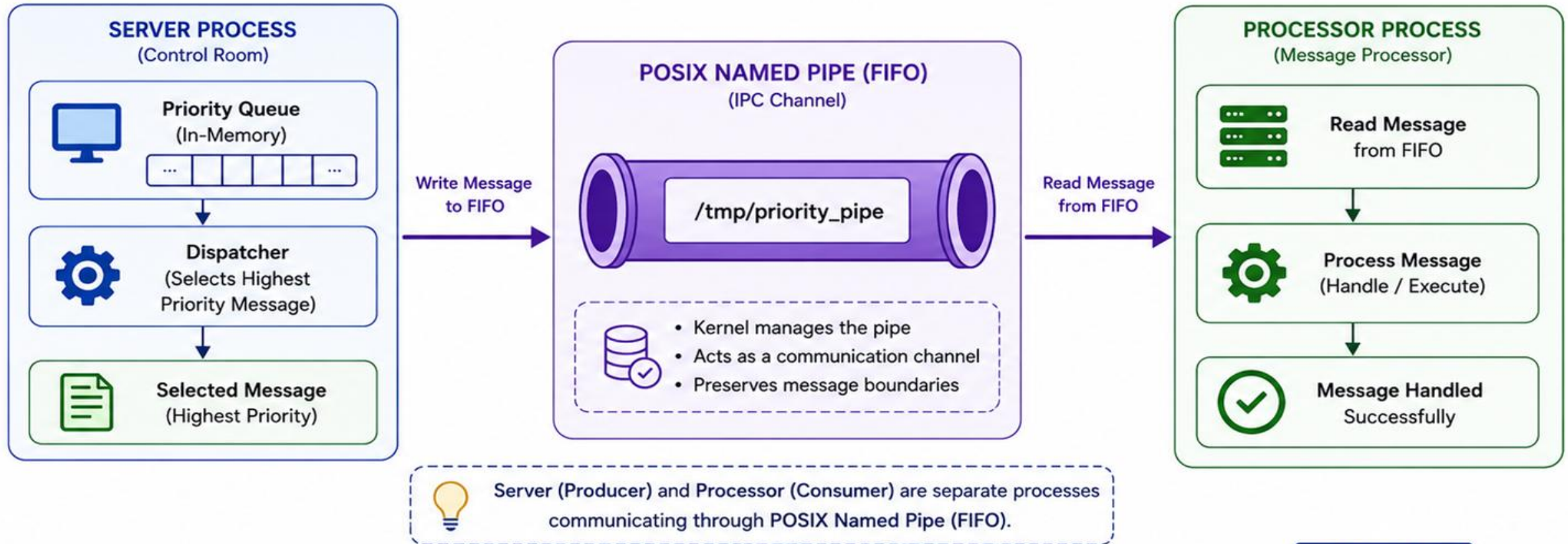
OPERATIONS SUPPORTED

- Enqueue (Insert)
- Dequeue (Remove Highest Priority)
- Peek (View Front)
- IsEmpty / IsFull
- Display Queue



IPC & MESSAGE PROCESSING

Using POSIX Named Pipes / FIFOs



HACKATHON STRESS TEST



100
Simultaneous
Stations



0
Message Loss



Bash script launched
100 clients simultaneously

```
for i in {1..100}; do ./client $i & done; wait
```



TEST SETUP



Bash script
launches 100
clients
simultaneously



100 clients
connect to
server via TCP



Server enqueues
messages in
Priority Queue



Dispatcher sends
highest-priority
messages to
Processor via FIFO



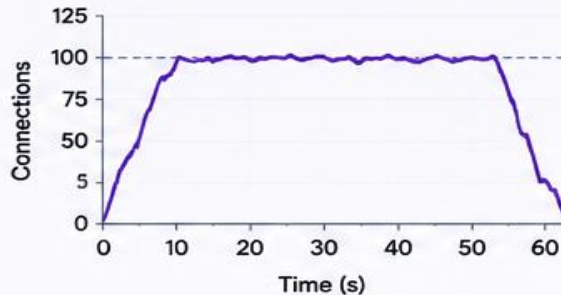
Processor reads
and processes
messages

TEST HIGHLIGHTS

- ✓ 0 message loss
- ✓ Tested concurrent TCP connections
- ✓ Validated pthread-based concurrency
- ✓ Correct severity ordering
- ✓ Correct timestamp / FIFO ordering
- ✓ Stable logging and IPC

CONCURRENCY OVER TIME

Active Connections Over Time

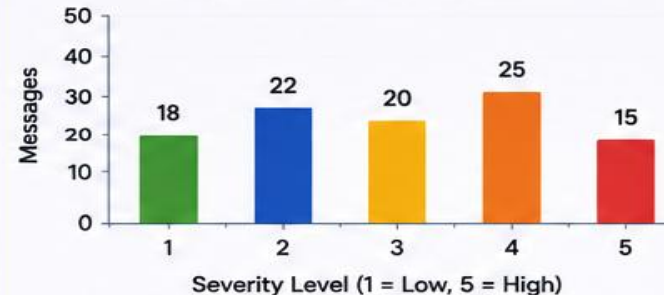


Summary

Peak Connections : 100
Total Messages : 100
Message Loss : 0
Test Duration : 60 sec
Result : **SUCCESS**

MESSAGE SEVERITY DISTRIBUTION

Messages by Severity Level (Total = 100)



LOG SNAPSHOT (SERVER)

```
[INFO] Server started...  
[INFO] Listening on port 5555  
[INFO] 100 clients connected  
[INFO] 100 messages enqueued successfully  
[INFO] Dispatcher running...  
[INFO] Messages sent to processor via FIFO  
[INFO] No message loss detected  
[INFO] Test completed successfully ✓
```

CONCLUSION & FUTURE SCOPE



ACHIEVEMENTS



Real-time TCP communication

Reliable and efficient communication between field stations and control room.



Concurrent client handling

Multiple clients handled simultaneously using pthread-based concurrency.



Priority-based scheduling

Messages prioritized by severity level to ensure critical alerts are processed first.



Thread-safe synchronization

Mutex and condition variables used to ensure safe access to shared resources.



Independent IPC processing

Server and processor run as separate processes using POSIX Named Pipes.



100-client stress validation

System tested with 100 simultaneous clients with 0 message loss and correct ordering.



FUTURE SCOPE



End-to-end encryption

Secure all communications using encryption to protect sensitive emergency data.



Geographic emergency mapping

Integrate GIS to visualize incidents and resources on real-time maps.



Database-backed persistence

Store messages, logs and user data in a database for reliability and scalability.



Fault-tolerant servers

Implement redundancy and failover mechanisms for high availability.



AI-based emergency prioritization

Use machine learning to predict and prioritize emergencies more accurately.



Multi-control-room deployment

Enable communication between multiple control rooms for large-scale operations.